COMPUTER NETWORKS LAB (MACSE513)

DA – 1

BY – AYAN CHAUDHURI (25MCS0038)

EXP 1: STUDY ABOUT NETWORKING COMMANDS

1. ping: Tests network connectivity to a destination host by sending ICMP Echo Request packets.

SYNTAX: ping <host_or_IP_address>
COMMAND: ping vit.ac.in

```
C:\Users\batch1>ping vit.ac.in

Pinging vit.ac.in [10.10.35.111] with 32 bytes of data:
Reply from 10.10.35.111: bytes=32 time=8ms TTL=61
Reply from 10.10.35.111: bytes=32 time<1ms TTL=61
Reply from 10.10.35.111: bytes=32 time=7ms TTL=61
Reply from 10.10.35.111: bytes=32 time=12ms TTL=61

Ping statistics for 10.10.35.111:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 12ms, Average = 6ms
```

2. netstat: Displays active TCP connections, listening ports, and network statistics.

SYNTAX: netstat [options] [interval]
COMMAND: netstat –a

```
Command Prompt

C:\Users\batch1>netstat -a

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:81             SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:135            SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:445            SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:3306           SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:3307           SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:3389           SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:5040           SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:5357           SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:5432           SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:5985           SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:8501           SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:29276          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:33060          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:33060          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:47001          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:48505          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:48506          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:48507          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:48508          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:48509          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:49664          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:49665          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:49666          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:49667          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:49668          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:58503          SJT622SCOPE041:0       LISTENING
  TCP    0.0.0.0:58504          SJT622SCOPE041:0       LISTENING
  TCP    10.30.164.51:139       SJT622SCOPE041:0       LISTENING
  TCP    10.30.164.51:48594     20.42.65.94:https      ESTABLISHED
  TCP    10.30.164.51:48595     104.208.16.90:https    ESTABLISHED
  TCP    10.30.164.51:48596     13.107.6.156:https     ESTABLISHED
  TCP    127.0.0.1:7680         SJT622SCOPE041:0       LISTENING
  TCP    127.0.0.1:11434        SJT622SCOPE041:0       LISTENING
  TCP    127.0.0.1:19293        SJT622SCOPE041:0       LISTENING
  TCP    127.0.0.1:22350        SJT622SCOPE041:0       LISTENING
  TCP    127.0.0.1:27017        SJT622SCOPE041:0       LISTENING
  TCP    127.0.0.1:29278        kubernetes:29279       ESTABLISHED
  TCP    127.0.0.1:29279        kubernetes:29278       ESTABLISHED
  TCP    127.0.0.1:29280        kubernetes:29281       ESTABLISHED
  TCP    127.0.0.1:29281        kubernetes:29280       ESTABLISHED
  TCP    127.0.0.1:29282        kubernetes:29283       ESTABLISHED
  TCP    127.0.0.1:29283        kubernetes:29282       ESTABLISHED
  TCP    127.0.0.1:31707        SJT622SCOPE041:0       LISTENING
  TCP    127.0.0.1:44950        SJT622SCOPE041:0       LISTENING
  TCP    127.0.0.1:44960        SJT622SCOPE041:0       LISTENING
```

3. tracert: Traces the route that packets take to reach a destination, showing each hop and its latency.

SYNTAX: tracert <target>
COMMAND: tracert vit.ac.in

```
C:\Users\batch1>tracert vit.ac.in

Tracing route to vit.ac.in [10.10.35.111]
over a maximum of 30 hops:

  1     1 ms     1 ms     1 ms  10.30.164.1
  2     1 ms     1 ms     1 ms  10.30.0.5
  3     1 ms    <1 ms    <1 ms  10.11.0.2
  4     8 ms    10 ms    13 ms  beta.vittbi.com [10.10.35.111]

Trace complete.
```

4. ipconfig: Displays current TCP/IP network configuration for all network adapters.

SYNTAX: ipconfig
COMMAND: ipconfig

```
C:\Users\batch1>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::33b7:bec0:e6a4:9907%6
   IPv4 Address. . . . . . . . . . . : 10.30.164.51
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.30.164.1
```

5. finger: The finger command is a command-line utility in Windows that is used to display information about users on a specified remote computer running the Finger service.

SYNTAX: finger [options] [user or user@host]
COMMAND: finger

6. nslookup: Queries DNS servers to resolve domain names to IP addresses and vice-versa.

SYNTAX: nslookup [options] [name/IP address] [DNS server]
COMMAND: nslookup vit.ac.in

```
C:\Users\batch1>nslookup vit.ac.in
Server:  umbrella2.vit.ac.in
Address:  10.10.4.10

Non-authoritative answer:
Name:    vit.ac.in
Address:  10.10.35.111
```

7. hostname: Displays the hostname of the local machine.

SYNTAX: hostname
COMMAND: hostname

```
C:\Users\batch1>hostname
SJT622SCOPE041
```

8. ARP: Displays the Address Resolution Protocol (ARP) cache, showing IP-to-MAC address mappings

SYNTAX: arp [options] [arguments]
COMMAND: arp -a

```
C:\Users\batch1>arp -a

Interface: 10.30.164.51 --- 0x6
  Internet Address      Physical Address      Type
  10.30.164.1           f8-e5-7e-bb-52-5d     dynamic
  10.30.164.11          38-ca-84-3c-c0-df     dynamic
  10.30.164.12          38-ca-84-3c-bf-b1     dynamic
  10.30.164.13          38-ca-84-3c-c2-ff     dynamic
  10.30.164.14          2c-58-b9-10-a1-30     dynamic
  10.30.164.15          38-ca-84-3c-c4-cf     dynamic
  10.30.164.16          38-ca-84-3c-c4-f5     dynamic
  10.30.164.17          38-ca-84-3c-c4-39     dynamic
  10.30.164.18          38-ca-84-3c-c2-18     dynamic
  10.30.164.19          38-ca-84-3c-c1-df     dynamic
  10.30.164.20          38-ca-84-3c-c1-76     dynamic
  10.30.164.21          2c-58-b9-10-a4-99     dynamic
  10.30.164.22          38-ca-84-3c-c4-f8     dynamic
  10.30.164.23          2c-58-b9-8c-8f-86     dynamic
  10.30.164.24          38-ca-84-3c-c0-70     dynamic
  10.30.164.25          38-ca-84-3c-c1-0f     dynamic
  10.30.164.26          38-ca-84-3c-c3-28     dynamic
  10.30.164.27          38-ca-84-3c-bf-88     dynamic
  10.30.164.28          2c-58-b9-8c-8e-11     dynamic
  10.30.164.29          38-ca-84-3c-c0-49     dynamic
  10.30.164.30          38-ca-84-3c-c4-ef     dynamic
  10.30.164.31          e0-73-e7-b1-26-18     dynamic
  10.30.164.32          38-ca-84-3c-c4-e3     dynamic
  10.30.164.33          38-ca-84-3c-c1-c6     dynamic
  10.30.164.34          38-ca-84-3c-c4-c9     dynamic
  10.30.164.35          38-ca-84-3c-c1-93     dynamic
  10.30.164.36          38-ca-84-3c-c0-8b     dynamic
  10.30.164.38          38-ca-84-3c-c1-83     dynamic
  10.30.164.39          38-ca-84-3c-c5-14     dynamic
  10.30.164.40          38-ca-84-3c-c7-05     dynamic
  10.30.164.41          38-ca-84-3c-c5-10     dynamic
  10.30.164.42          2c-58-b9-10-a6-6f     dynamic
  10.30.164.43          38-ca-84-3c-c4-af     dynamic
  10.30.164.44          38-ca-84-3c-94-81     dynamic
  10.30.164.45          38-ca-84-3c-c0-ae     dynamic
  10.30.164.46          10-b6-76-4d-9c-61     dynamic
  10.30.164.47          38-ca-84-3c-be-07     dynamic
  10.30.164.48          38-ca-84-3c-c1-25     dynamic
  10.30.164.49          38-ca-84-3c-c5-32     dynamic
  10.30.164.50          38-ca-84-3c-c3-48     dynamic
  10.30.164.52          38-ca-84-3c-c3-14     dynamic
  10.30.164.53          38-ca-84-3c-c2-b6     dynamic
  10.30.164.54          38-ca-84-3c-c3-68     dynamic
  10.30.164.55          38-ca-84-3c-c0-c3     dynamic
  10.30.164.56          38-ca-84-3c-c5-12     dynamic
  10.30.164.57          38-ca-84-3c-ac-f5     dynamic
```

9. getmac: Displays the MAC address(es) of the network adapter(s).

SYNTAX: : getmac
COMMAND: getmac

```
C:\Users\batch1>getmac

Physical Address    Transport Name
=================== ==========================================================
38-CA-84-3C-C4-60   \Device\Tcpip_{27042B61-FDE3-4B21-9202-8B4F9CA11438}
```

10. route print: Displays the IP routing table

SYNTAX: route print
COMMAND: route print

```
C:\Users\batch1>route print
===========================================================================
Interface List
  6...38 ca 84 3c c4 60 ......Intel(R) Ethernet Connection (17) I219-LM
  1...........................Software Loopback Interface 1
===========================================================================

IPv4 Route Table
===========================================================================
Active Routes:
Network Destination        Netmask          Gateway       Interface  Metric
          0.0.0.0          0.0.0.0      10.30.164.1     10.30.164.51    281
      10.30.164.0    255.255.255.0         On-link      10.30.164.51    281
     10.30.164.51  255.255.255.255         On-link      10.30.164.51    281
    10.30.164.255  255.255.255.255         On-link      10.30.164.51    281
        127.0.0.0        255.0.0.0         On-link         127.0.0.1    331
        127.0.0.1  255.255.255.255         On-link         127.0.0.1    331
  127.255.255.255  255.255.255.255         On-link         127.0.0.1    331
        224.0.0.0        240.0.0.0         On-link         127.0.0.1    331
        224.0.0.0        240.0.0.0         On-link      10.30.164.51    281
  255.255.255.255  255.255.255.255         On-link         127.0.0.1    331
  255.255.255.255  255.255.255.255         On-link      10.30.164.51    281
===========================================================================
Persistent Routes:
  Network Address          Netmask  Gateway Address  Metric
          0.0.0.0          0.0.0.0      10.30.164.1  Default
===========================================================================

IPv6 Route Table
===========================================================================
Active Routes:
 If Metric Network Destination      Gateway
  1    331 ::1/128                  On-link
  6    281 fe80::/64                On-link
  6    281 fe80::33b7:bec0:e6a4:9907/128
                                    On-link
  1    331 ff00::/8                 On-link
  6    281 ff00::/8                 On-link
===========================================================================
Persistent Routes:
  None
```

11. whoami: The whoami command shows information about the user, groups, and privileges for the account currently logged on to the local system

SYNTAX: whoami
COMMAND: whoami

```
C:\Users\batch1>whoami
vituniversity\batch1
```

12. Telnet: Used to establish a connection to a remote host, often to test if a specific port is open (requires Telnet Client feature to be enabled).

SYNTAX: telnet [hostname or IP address] [port number]
COMMAND: telnet 10.10.5.21

```
Telnet 10.10.5.21

CentOS release 5.4 (Final)
Kernel 2.6.18-164.el5PAE on an i686
login: 25MCS0038
Password:
Last login: Wed Dec 17 12:07:49 from 10.30.164.51
[25MCS0038@VITLINX ~]$
```

13. systeminfo: Displays detailed system information, including network configuration details

SYNTAX: systeminfo
COMMAND: systeminfo

```
C:\Users\batch1>systeminfo

Host Name:                 SJT622SCOPE041
OS Name:                   Microsoft Windows 11 Pro
OS Version:                10.0.22631 N/A Build 22631
OS Manufacturer:           Microsoft Corporation
OS Configuration:          Member Workstation
OS Build Type:             Multiprocessor Free
Registered Owner:          admin
Registered Organization:   HP Inc.
Product ID:                00355-60817-89306-AAOEM
Original Install Date:     28-07-2023, 14:10:31
System Boot Time:          10-12-2025, 13:51:10
System Manufacturer:       HP
System Model:              HP Z2 SFF G9 Workstation Desktop PC
System Type:               x64-based PC
Processor(s):              1 Processor(s) Installed.
                           [01]: Intel64 Family 6 Model 151 Stepping 2 GenuineIntel ~2400 Mhz
BIOS Version:              HP U50 Ver. 01.03.03, 29-08-2022
Windows Directory:         C:\WINDOWS
System Directory:          C:\WINDOWS\system32
Boot Device:               \Device\HarddiskVolume1
System Locale:             en-us;English (United States)
Input Locale:              00004009
Time Zone:                 (UTC+05:30) Chennai, Kolkata, Mumbai, New Delhi
Total Physical Memory:     32,473 MB
Available Physical Memory: 23,722 MB
Virtual Memory: Max Size:  37,337 MB
Virtual Memory: Available: 27,035 MB
Virtual Memory: In Use:    10,302 MB
Page File Location(s):     C:\pagefile.sys
Domain:                    VITUNIVERSITY.LOCAL
Logon Server:              \\VITCTS-BDC
Hotfix(s):                 4 Hotfix(s) Installed.
                           [01]: KB5049624
                           [02]: KB5027397
                           [03]: KB5053602
                           [04]: KB5052107
Network Card(s):           1 NIC(s) Installed.
                           [01]: Intel(R) Ethernet Connection (17) I219-LM
                                 Connection Name: Ethernet
                                 DHCP Enabled:    No
                                 IP address(es)
                                 [01]: 10.30.164.51
                                 [02]: fe80::33b7:bec0:e6a4:9907
Hyper-V Requirements:      A hypervisor has been detected. Features required for Hyper-V will not be displayed.
```

14. pathping: Combines features of ping and tracert, providing detailed statistics on packet loss at each hop.

SYNTAX: pathping <target_name>
COMMAND: pathping vit.ac.in

```
C:\Users\batch1>pathping vit.ac.in

Tracing route to vit.ac.in [10.10.35.111]
over a maximum of 30 hops:
  0  host.docker.internal [10.30.164.51]
  1  10.30.164.1
  2  10.30.0.5
  3  10.11.0.2
  4  vittbi.com [10.10.35.111]

Computing statistics for 100 seconds...
            Source to Here   This Node/Link
Hop  RTT    Lost/Sent = Pct  Lost/Sent = Pct  Address
  0                                           host.docker.internal [10.30.164.51]
                                     0/ 100 =  0%   |
  1    3ms     0/ 100 =  0%     0/ 100 =  0%  10.30.164.1
                                     0/ 100 =  0%   |
  2    2ms     0/ 100 =  0%     0/ 100 =  0%  10.30.0.5
                                     0/ 100 =  0%   |
  3    2ms     0/ 100 =  0%     0/ 100 =  0%  10.11.0.2
                                     1/ 100 =  1%   |
  4    2ms     1/ 100 =  1%     0/ 100 =  0%  vittbi.com [10.10.35.111]

Trace complete.
```

15. Tasklist: Display running processes and there network usage

SYNTAX: tasklist
COMMAND: tasklist

```
C:\Users\batch1>tasklist

Image Name                     PID Session Name        Session#    Mem Usage
========================= ======== ================ =========== ============
System Idle Process              0 Services                   0          8 K
System                           4 Services                   0      5,724 K
Secure System                  236 Services                   0     75,812 K
Registry                       292 Services                   0     67,456 K
smss.exe                       832 Services                   0      1,296 K
csrss.exe                     1056 Services                   0      6,272 K
wininit.exe                   1296 Services                   0      7,480 K
csrss.exe                     1304 Console                    1      6,116 K
services.exe                  1368 Services                   0     20,524 K
LsaIso.exe                    1388 Services                   0      4,336 K
lsass.exe                     1404 Services                   0     31,456 K
winlogon.exe                  1484 Console                    1     11,112 K
svchost.exe                   1620 Services                   0     34,244 K
WUDFHost.exe                  1652 Services                   0      9,160 K
fontdrvhost.exe               1696 Services                   0      4,360 K
fontdrvhost.exe               1704 Console                    1     11,996 K
svchost.exe                   1796 Services                   0     19,364 K
svchost.exe                   1868 Services                   0     11,680 K
svchost.exe                   1976 Services                   0      8,476 K
svchost.exe                   1984 Services                   0     13,644 K
svchost.exe                   1992 Services                   0      5,656 K
svchost.exe                   2004 Services                   0     11,552 K
svchost.exe                   2012 Services                   0      7,912 K
svchost.exe                   2020 Services                   0      6,572 K
svchost.exe                   1428 Services                   0      8,592 K
svchost.exe                   2056 Services                   0     10,088 K
svchost.exe                   2092 Services                   0     11,488 K
svchost.exe                   2108 Services                   0     10,804 K
svchost.exe                   2228 Services                   0     58,152 K
IntelCpHDCPSvc.exe            2236 Services                   0      5,876 K
svchost.exe                   2272 Services                   0      6,060 K
svchost.exe                   2316 Services                   0     18,332 K
dwm.exe                       2504 Console                    1  1,01,436 K
svchost.exe                   2564 Services                   0     18,624 K
svchost.exe                   2608 Services                   0      7,800 K
svchost.exe                   2832 Services                   0     11,244 K
svchost.exe                   2928 Services                   0      7,008 K
NVDisplay.Container.exe       2940 Services                   0     22,588 K
svchost.exe                   3052 Services                   0      8,068 K
svchost.exe                   2484 Services                   0     11,212 K
svchost.exe                   3160 Services                   0     11,308 K
svchost.exe                   3172 Services                   0     10,276 K
k7tsmngr.exe                  3180 Services                   0      2,092 K
svchost.exe                   3232 Services                   0     11,816 K
svchost.exe                   3396 Services                   0     10,736 K
svchost.exe                   3460 Services                   0      6,640 K
```

16.gpresult: Displays the Group Policy settings applied to the machine/user

SYNTAX: gpresult [options]
COMMAND: gpresult /r

```
C:\Users\batch1>gpresult /r

Microsoft (R) Windows (R) Operating System Group Policy Result tool v2.0
© Microsoft Corporation. All rights reserved.

Created on  17- 12- 2025 at 14:33:04


RSOP data for VITUNIVERSITY\Batch1 on SJT622SCOPE041 : Logging Mode
---------------------------------------------------------------------

OS Configuration:            Member Workstation
OS Version:                  10.0.22631
Site Name:                   N/A
Roaming Profile:             N/A
Local Profile:               C:\Users\batch1
Connected over a slow link?: No


USER SETTINGS
-------------
    CN=Batch1,OU=Batch-ExamUsers,DC=VITUNIVERSITY,DC=LOCAL
    Last time Group Policy was applied: 17-12-2025 at 13:00:47
    Group Policy was applied from:      CDC-ADC.VITUNIVERSITY.LOCAL
    Group Policy slow link threshold:   500 kbps
    Domain Name:                        VITUNIVERSITY
    Domain Type:                        Windows 2008 or later

    Applied Group Policy Objects
    ----------------------------
        Local Group Policy

    The user is a part of the following security groups
    ---------------------------------------------------
        Domain Users
        Everyone
        BUILTIN\Users
        NT AUTHORITY\INTERACTIVE
        CONSOLE LOGON
        NT AUTHORITY\Authenticated Users
        This Organization
        LOCAL
        vdi.sce
        vdi.vgpu7
        vdi.vgpu-users
        ftpgroups
        Authentication authority asserted identity
        Medium Mandatory Level
```

17. netsh: Lists all the saved Wifi network profiles on the system

SYNTAX: netsh [options] [context] [subcontext ...] [command] [parameters]
COMMAND: netsh wlan show profiles

```
C:\Users\batch1>netsh wlan show profiles
There is no wireless interface on the system.
```

18. Get-NetIPAddress: Provides a comprehensive list of all IP addresses (IPv4 and IPv6) assigned to assigned to network adapter.

SYNTAX: Get-NetIPAddress
COMMAND: Get-NetIPAddress

```
PS C:\Users\batch1> Get-NetIPAddress


IPAddress         : fe80::33b7:bec0:e6a4:9907%6
InterfaceIndex    : 6
InterfaceAlias    : Ethernet
AddressFamily     : IPv6
Type              : Unicast
PrefixLength      : 64
PrefixOrigin      : WellKnown
SuffixOrigin      : Link
AddressState      : Preferred
ValidLifetime     :
PreferredLifetime :
SkipAsSource      : False
PolicyStore       : ActiveStore

IPAddress         : ::1
InterfaceIndex    : 1
InterfaceAlias    : Loopback Pseudo-Interface 1
AddressFamily     : IPv6
Type              : Unicast
PrefixLength      : 128
PrefixOrigin      : WellKnown
SuffixOrigin      : WellKnown
AddressState      : Preferred
ValidLifetime     :
PreferredLifetime :
SkipAsSource      : False
PolicyStore       : ActiveStore

IPAddress         : 10.30.164.51
InterfaceIndex    : 6
InterfaceAlias    : Ethernet
AddressFamily     : IPv4
Type              : Unicast
PrefixLength      : 24
PrefixOrigin      : Manual
SuffixOrigin      : Manual
AddressState      : Preferred
ValidLifetime     :
PreferredLifetime :
SkipAsSource      : False
PolicyStore       : ActiveStore

IPAddress         : 127.0.0.1
InterfaceIndex    : 1
InterfaceAlias    : Loopback Pseudo-Interface 1
AddressFamily     : IPv4
Type              : Unicast
PrefixLength      : 8
PrefixOrigin      : WellKnown
SuffixOrigin      : WellKnown
AddressState      : Preferred
ValidLifetime     :
PreferredLifetime :
SkipAsSource      : False
PolicyStore       : ActiveStore
```

19. Get-NetAdapter: Lists all hardware network adapter

SYNTAX: Get-NetAdapter
COMMAND: Get-NetAdapter

```
PS C:\Users\batch1> Get-NetAdapter

Name                      InterfaceDescription                    ifIndex Status       MacAddress             LinkSpeed
----                      --------------------                    ------- ------       ----------             ---------
Ethernet                  Intel(R) Ethernet Connection (17) I2...       6 Up           38-CA-84-3C-C4-60         1 Gbps
```

20. nbstat: Used to display NetBIOS over TCP/IP protocol statistics, NetBIOS name Tables and the NetBIOS name cache.

SYNTAX: nbtstat [options] [interval]
COMMAND: nbtstat –n

```
PS C:\Users\batch1> nbtstat -n

Ethernet:
Node IpAddress: [10.30.164.51] Scope Id: []

                NetBIOS Local Name Table

       Name               Type         Status
    ---------------------------------------------
    SJT622SCOPE041 <20>  UNIQUE      Registered
    SJT622SCOPE041 <00>  UNIQUE      Registered
    VITUNIVERSITY  <00>  GROUP       Registered
PS C:\Users\batch1>
```

EXP 2: SOCKET PROGRAMMING

(A) CLIENT SERVER COMMUNICATION USING UDP

ALGRITHM-

Server Side

1. Start
2. Set HOST = 127.0.0.1 and PORT = 9999
3. Create a UDP socket: sock = socket(AF_INET, SOCK_DGRAM)
4. Bind the socket to (HOST, PORT) so it can receive messages
5. Display: “UDP Server listening on HOST:PORT”
6. Repeat forever:
    - Receive a UDP packet using recvfrom(2048) → returns (data, addr)
    - Decode data from bytes to string using UTF-8
    - Print: Received from addr: message
    - Create reply text: "ACK: " + message
    - Encode reply to bytes (UTF-8)
    - Send reply back to the same client address using sendto(reply_bytes, addr)
    - Stop (only if program is manually terminated)

Client Side

1. Start
2. Set SERVER_HOST = 127.0.0.1 and SERVER_PORT = 9999
3. Create a UDP socket: sock = socket(AF_INET, SOCK_DGRAM)
4. Set receive timeout to 3 seconds: sock.settimeout(3)
5. Display prompt: “Type messages…”
6. Repeat forever:
    - Read a message from user: msg = input()
    - Convert message to bytes (UTF-8)
    - Send message to server using sendto(msg_bytes, (SERVER_HOST, SERVER_PORT))
    - Try to receive reply using recvfrom(2048):
        - If reply received within 3 sec:
            - Decode reply bytes to text (UTF-8)
            - Print: Server: <reply>
        - If timeout occurs:
            - Print: “No response (timeout). Is the server running?”
7. Stop (only when user terminates program, e.g., Ctrl+C)

CODE:

udp_client.py

```
import socket

SERVER_HOST = "127.0.0.1"
SERVER_PORT = 9999

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.settimeout(3)

print("Type messages. Ctrl+C to exit.")

while True:
    msg = input("You: ")
    sock.sendto(msg.encode("utf-8"), (SERVER_HOST, SERVER_PORT))
```

```
    try:
        data, addr = sock.recvfrom(2048)
        print("Server:", data.decode("utf-8", errors="replace"))
    except socket.timeout:
        print("No response (timeout). Is the server running?")
```

udp_server.py

```
import socket

HOST = "127.0.0.1"   # listen on localhost inside WSL
PORT = 9999          # UDP port

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((HOST, PORT))

print(f"UDP Server listening on {HOST}:{PORT}")

while True:
    data, addr = sock.recvfrom(2048)  # (bytes, (ip, port))
    msg = data.decode("utf-8", errors="replace")
    print(f"Received from {addr}: {msg}")

    reply = f"ACK: {msg}"
    sock.sendto(reply.encode("utf-8"), addr)
```

OUTPUT:

```
root@BOOK-27C8PD5KBQ:~# mkdir -p ~/udp_socket_demo
root@BOOK-27C8PD5KBQ:~# cd ~/udp_socket_demo
root@BOOK-27C8PD5KBQ:~/udp_socket_demo# nano udp_server.py
root@BOOK-27C8PD5KBQ:~/udp_socket_demo# nano udp_client.py
root@BOOK-27C8PD5KBQ:~/udp_socket_demo# python3 udp_server.py
UDP Server listening on 127.0.0.1:9999
Received from ('127.0.0.1', 50552): hi
Received from ('127.0.0.1', 50552): how are you
```

```
root@BOOK-27C8PD5KBQ:~# cd ~/udp_socket_demo
root@BOOK-27C8PD5KBQ:~/udp_socket_demo# python3 udp_client.py
Type messages. Ctrl+C to exit.
You: hi
Server: ACK: hi
You: how are you
Server: ACK: how are you
You: ^CTraceback (most recent call last):
  File "/root/udp_socket_demo/udp_client.py", line 12, in <module>
    msg = input("You: ")
          ^^^^^^^^^^^^^^
KeyboardInterrupt

root@BOOK-27C8PD5KBQ:~/udp_socket_demo#
```

(B) SIMPLE CHAT APPLICATION USING TCP

ALGORITHM:

Server Side:

1. Start
2. Declare variables: server_fd, client_fd, addresses (server_addr, client_addr), buffers (send_buf, recv_buf)
3. Create TCP socket
4. Set socket option

5. Prepare server address
    - Set sin_family = AF_INET
    - Set sin_addr = 127.0.0.1 (localhost)
    - Set sin_port = htons(8080)
6. Bind socket to IP and port
7. Listen for incoming connection
8. Accept a client connection
9. Print: Client connected and instructions
10. Repeat until chat ends
11. Close sockets
12. Stop

Client Side:

1. Start
2. Declare variables: sock, server_addr, buffers (send_buf, recv_buf)
3. Create TCP socket
4. Prepare server address
    - Set sin_family = AF_INET
    - Set sin_port = htons(8080)
    - Convert IP "127.0.0.1" to binary using inet_pton
    - If conversion fails → print error, close socket, stop
5. Connect to server
6. Print: connected + instructions (exit to close)
7. Repeat until chat ends
8. Close socket
9. Stop

CODE:

chat server

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUF_SIZE 1024

int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_len = sizeof(client_addr);

    char send_buf[BUF_SIZE], recv_buf[BUF_SIZE];

    // 1) Create socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return 1;
    }

    // (Optional) Reuse address to avoid "Address already in use"
    int opt = 1;
```

```
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

// 2) Bind
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1"); // localhost (WSL)
server_addr.sin_port = htons(PORT);

if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
    perror("bind");
    close(server_fd);
    return 1;
}

// 3) Listen
if (listen(server_fd, 1) < 0) {
    perror("listen");
    close(server_fd);
    return 1;
}

printf("Server listening on 127.0.0.1:%d...\n", PORT);

// 4) Accept
client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &client_len);
if (client_fd < 0) {
    perror("accept");
    close(server_fd);
    return 1;
}

printf("Client connected!\n");
printf("Type 'exit' to close.\n");

while (1) {
    // Receive from client
    memset(recv_buf, 0, BUF_SIZE);
    int n = recv(client_fd, recv_buf, BUF_SIZE - 1, 0);
    if (n <= 0) {
        printf("Client disconnected.\n");
        break;
    }
    recv_buf[n] = '\0';
    printf("Client: %s\n", recv_buf);

    if (strncmp(recv_buf, "exit", 4) == 0) {
        printf("Client ended chat.\n");
        break;
    }

    // Send to client
    printf("You (Server): ");
    fflush(stdout);

    if (!fgets(send_buf, BUF_SIZE, stdin)) break;
    send_buf[strcspn(send_buf, "\n")] = '\0'; // remove newline

    if (send(client_fd, send_buf, strlen(send_buf), 0) < 0) {
```

```
            perror("send");
            break;
        }

        if (strncmp(send_buf, "exit", 4) == 0) {
            printf("Server ended chat.\n");
            break;
        }
    }

    close(client_fd);
    close(server_fd);
    return 0;
}
```

chat client

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUF_SIZE 1024

int main() {
    int sock;
    struct sockaddr_in server_addr;
    char send_buf[BUF_SIZE], recv_buf[BUF_SIZE];

    // 1) Create socket
    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("socket");
        return 1;
    }

    // 2) Server address
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);

    // connect to localhost (WSL)
    if (inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr) <= 0) {
        perror("inet_pton");
        close(sock);
        return 1;
    }

    // 3) Connect
    if (connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("connect");
        close(sock);
        return 1;
    }

    printf("Connected to server 127.0.0.1:%d\n", PORT);
```

```
    printf("Type 'exit' to close.\n");

    while (1) {
        // Send message to server
        printf("You (Client): ");
        fflush(stdout);

        if (!fgets(send_buf, BUF_SIZE, stdin)) break;
        send_buf[strcspn(send_buf, "\n")] = '\0';

        if (send(sock, send_buf, strlen(send_buf), 0) < 0) {
            perror("send");
            break;
        }

        if (strncmp(send_buf, "exit", 4) == 0) {
            printf("Client ended chat.\n");
            break;
        }

        // Receive server reply
        memset(recv_buf, 0, BUF_SIZE);
        int n = recv(sock, recv_buf, BUF_SIZE - 1, 0);
        if (n <= 0) {
            printf("Server disconnected.\n");
            break;
        }
        recv_buf[n] = '\0';
        printf("Server: %s\n", recv_buf);

        if (strncmp(recv_buf, "exit", 4) == 0) {
            printf("Server ended chat.\n");
            break;
        }
    }

    close(sock);
    return 0;
}
```

OUTPUT

```
root@BOOK-27C8PD5KBQ:~# gcc server.c -o server
cc1: fatal error: server.c: No such file or directory
compilation terminated.
root@BOOK-27C8PD5KBQ:~# cd ~/tcp_chat_c
root@BOOK-27C8PD5KBQ:~/tcp_chat_c# gcc server.c -o server
root@BOOK-27C8PD5KBQ:~/tcp_chat_c# gcc client.c -o client
root@BOOK-27C8PD5KBQ:~/tcp_chat_c# ./server
Server listening on 127.0.0.1:8080...
Client connected!
Type 'exit' to close.
Client: HI
You (Server): HOW ARE YOU?
Client: I AM FINE, HOW ARE YOU?
You (Server): I AM FINE THANKS
```

```
root@BOOK-27C8PD5KBQ:~# cd ~/tcp_chat_c
root@BOOK-27C8PD5KBQ:~/tcp_chat_c# ./client
Connected to server 127.0.0.1:8080
Type 'exit' to close.
You (Client): HI
Server: HOW ARE YOU?
You (Client): I AM FINE, HOW ARE YOU?
Server: I AM FINE THANKS
You (Client):
```

(C) FILE SENDING CHAT APPLICATION USING TCP

ALGORITHM

Server Side:

1. Start
2. Create control TCP listener on given port using make_listener()
3. Accept client using accept_one()
4. Send greeting: 220 Welcome...
5. Repeat forever using select() to monitor:
   - Client socket (incoming commands)
   - Server keyboard (STDIN) (server chat input)
6. If client socket is ready:
   - Read one full line using recv_line()
   - If client disconnected → stop
   - Parse command:
     - MSG <text> → display client chat on server screen
     - STOR <name> <size> (upload):
     - RETR <name> (download):
     - QUIT → send 221 Bye and stop
     - Otherwise → print unknown command
7. If server STDIN is ready:
   - Read line from keyboard
   - If /quit → send QUIT to client and stop
   - Else send as chat: MSG <line>
8. Close sockets (client + listener)
9. Stop

Client Side:

1. Start
2. Read server_ip and port from command line
3. Connect to server using connect_tcp(server_ip, port) → control connection
4. Receive greeting using recv_line()
5. Repeat forever using select() to monitor:
6. If control socket is ready:
   - Read one line using recv_line()
   - If server disconnected → stop

- If line starts with MSG → print server chat
- Else print status (220/226/5xx etc.)

7. If STDIN is ready:
    - Read line from keyboard
    - If /quit → send QUIT and stop
    - If /put <path> (upload)
    - If /get <name> (download):
    - Else (normal text) → send MSG <text>
8. Close control socket
9. Stop

CODE:
Server chat

```
#define _POSIX_C_SOURCE 200809L
#include <arpa/inet.h>
#include <errno.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/select.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#define LINE_MAX 4096
#define DATA_BUFSZ 8192

static void die(const char *msg) {
    perror(msg);
    exit(1);
}

static int send_all(int fd, const void *buf, size_t len) {
    const char *p = (const char*)buf;
    size_t sent = 0;
    while (sent < len) {
        ssize_t n = send(fd, p + sent, len - sent, 0);
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        sent += (size_t)n;
    }
    return 0;
}

static int send_line(int fd, const char *line) {
    return send_all(fd, line, strlen(line));
}

// Read a line ending in '\n' from socket fd into out (null-terminated).
// Returns: 1 = success, 0 = peer closed, -1 = error, -2 = line too long
static int recv_line(int fd, char *out, size_t out_sz) {
```

```
    size_t used = 0;
    while (1) {
        if (used + 1 >= out_sz) return -2;
        char c;
        ssize_t n = recv(fd, &c, 1, 0);
        if (n == 0) return 0;
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        out[used++] = c;
        out[used] = '\0';
        if (c == '\n') return 1;
    }
}

// Very basic filename safety: only allow plain names (no /, \, ..)
static int safe_filename(const char *name) {
    if (!name || !*name) return 0;
    if (strstr(name, "..")) return 0;
    if (strchr(name, '/')) return 0;
    if (strchr(name, '\\')) return 0;
    return 1;
}

static int make_listener(uint16_t port, uint16_t *out_port) {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    if (s < 0) return -1;

    int yes = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(port);

    if (bind(s, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        close(s);
        return -1;
    }
    if (listen(s, 1) < 0) {
        close(s);
        return -1;
    }

    if (out_port) {
        socklen_t alen = sizeof(addr);
        if (getsockname(s, (struct sockaddr*)&addr, &alen) == 0) {
            *out_port = ntohs(addr.sin_port);
        } else {
            *out_port = port;
        }
    }
    return s;
}
```

```
static int accept_one(int listener) {
    struct sockaddr_in caddr;
    socklen_t clen = sizeof(caddr);
    int c = accept(listener, (struct sockaddr*)&caddr, &clen);
    return c;
}

static int handle_stor(int control_fd, const char *filename, long long size) {
    if (!safe_filename(filename) || size < 0) {
        send_line(control_fd, "550 Invalid filename/size\n");
        return 0;
    }

    mkdir("server_files", 0755);

    // Create PASV data listener on ephemeral port (0).
    uint16_t pasv_port = 0;
    int data_listener = make_listener(0, &pasv_port);
    if (data_listener < 0) {
        send_line(control_fd, "425 Can't open data connection\n");
        return 0;
    }

    char resp[128];
    snprintf(resp, sizeof(resp), "150 PASV %u\n", pasv_port);
    if (send_line(control_fd, resp) < 0) {
        close(data_listener);
        return 0;
    }

    int data_fd = accept_one(data_listener);
    close(data_listener);
    if (data_fd < 0) {
        send_line(control_fd, "425 Data accept failed\n");
        return 0;
    }

    char path[512];
    snprintf(path, sizeof(path), "server_files/%s", filename);
    FILE *fp = fopen(path, "wb");
    if (!fp) {
        close(data_fd);
        send_line(control_fd, "550 Can't open file for write\n");
        return 0;
    }

    long long remaining = size;
    char buf[DATA_BUFSZ];

    while (remaining > 0) {
        ssize_t chunk = (remaining > DATA_BUFSZ) ? DATA_BUFSZ : (ssize_t)remaining;
        ssize_t n = recv(data_fd, buf, chunk, 0);
        if (n <= 0) {
            fclose(fp);
            close(data_fd);
            send_line(control_fd, "426 Connection closed; transfer aborted\n");
```

```
            return 0;
        }
        fwrite(buf, 1, (size_t)n, fp);
        remaining -= n;
    }

    fclose(fp);
    close(data_fd);

    snprintf(resp, sizeof(resp), "226 Stored %s\n", filename);
    send_line(control_fd, resp);
    return 1;
}

static int handle_retr(int control_fd, const char *filename) {
    if (!safe_filename(filename)) {
        send_line(control_fd, "550 Invalid filename\n");
        return 0;
    }

    char path[512];
    snprintf(path, sizeof(path), "server_files/%s", filename);

    FILE *fp = fopen(path, "rb");
    if (!fp) {
        send_line(control_fd, "550 File not found\n");
        return 0;
    }

    // Find file size
    if (fseek(fp, 0, SEEK_END) != 0) {
        fclose(fp);
        send_line(control_fd, "550 Can't read file\n");
        return 0;
    }
    long long size = ftell(fp);
    fseek(fp, 0, SEEK_SET);

    uint16_t pasv_port = 0;
    int data_listener = make_listener(0, &pasv_port);
    if (data_listener < 0) {
        fclose(fp);
        send_line(control_fd, "425 Can't open data connection\n");
        return 0;
    }

    char resp[160];
    snprintf(resp, sizeof(resp), "150 PASV %u %lld\n", pasv_port, size);
    if (send_line(control_fd, resp) < 0) {
        fclose(fp);
        close(data_listener);
        return 0;
    }

    int data_fd = accept_one(data_listener);
    close(data_listener);
    if (data_fd < 0) {
```

```
        fclose(fp);
        send_line(control_fd, "425 Data accept failed\n");
        return 0;
    }

    char buf[DATA_BUFSZ];
    long long remaining = size;
    while (remaining > 0) {
        size_t r = fread(buf, 1, (remaining > DATA_BUFSZ) ? DATA_BUFSZ : (size_t)remaining, fp);
        if (r == 0) break;
        if (send_all(data_fd, buf, r) < 0) {
            fclose(fp);
            close(data_fd);
            send_line(control_fd, "426 Connection closed; transfer aborted\n");
            return 0;
        }
        remaining -= (long long)r;
    }

    fclose(fp);
    close(data_fd);

    snprintf(resp, sizeof(resp), "226 Sent %s\n", filename);
    send_line(control_fd, resp);
    return 1;
}

int main(int argc, char **argv) {
    uint16_t port = 2121;
    if (argc >= 2) port = (uint16_t)atoi(argv[1]);

    int listener = make_listener(port, NULL);
    if (listener < 0) die("listen socket");

    printf("Server listening on 0.0.0.0:%u\n", port);
    printf("Chat: type and press Enter to send.\n");
    printf("Commands received from client:\n");
    printf("  MSG <text>\\n\n  STOR <name> <size>\\n\n  RETR <name>\\n\n  QUIT\\n\n");

    int client = accept_one(listener);
    if (client < 0) die("accept");
    printf("Client connected.\n");

    // Send greeting
    send_line(client, "220 Welcome to mini-FTP-chat server\n");

    fd_set rfds;
    char line[LINE_MAX];

    while (1) {
        FD_ZERO(&rfds);
        FD_SET(client, &rfds);
        FD_SET(STDIN_FILENO, &rfds);

        int maxfd = (client > STDIN_FILENO) ? client : STDIN_FILENO;
        int r = select(maxfd + 1, &rfds, NULL, NULL, NULL);
        if (r < 0) {
```

```
    if (errno == EINTR) continue;
    die("select");
}

// Incoming from client
if (FD_ISSET(client, &rfds)) {
    int rr = recv_line(client, line, sizeof(line));
    if (rr == 0) {
        printf("Client disconnected.\n");
        break;
    }
    if (rr < 0) {
        printf("recv_line error.\n");
        break;
    }

    // Parse commands
    if (strncmp(line, "MSG ", 4) == 0) {
        printf("Client: %s", line + 4);
    } else if (strncmp(line, "STOR ", 5) == 0) {
        char fname[256];
        long long fsz;
        if (sscanf(line + 5, "%255s %lld", fname, &fsz) == 2) {
            printf("Client wants upload: %s (%lld bytes)\n", fname, fsz);
            handle_stor(client, fname, fsz);
        } else {
            send_line(client, "500 Bad STOR syntax\n");
        }
    } else if (strncmp(line, "RETR ", 5) == 0) {
        char fname[256];
        if (sscanf(line + 5, "%255s", fname) == 1) {
            printf("Client wants download: %s\n", fname);
            handle_retr(client, fname);
        } else {
            send_line(client, "500 Bad RETR syntax\n");
        }
    } else if (strncmp(line, "QUIT", 4) == 0) {
        send_line(client, "221 Bye\n");
        printf("Client quit.\n");
        break;
    } else {
        // Unknown line; print it
        printf("Client (raw): %s", line);
    }
}

// Server operator typing -> send chat to client
if (FD_ISSET(STDIN_FILENO, &rfds)) {
    if (!fgets(line, sizeof(line), stdin)) break;

    // Server supports local commands too:
    // /quit, /put, /get handled on client side only (server is authoritative)
    if (strncmp(line, "/quit", 5) == 0) {
        send_line(client, "QUIT\n");
        break;
    }
```

```
            // Default: send as chat
            char out[LINE_MAX + 8];
            snprintf(out, sizeof(out), "MSG %s", line);
            if (send_line(client, out) < 0) {
                printf("Send failed.\n");
                break;
            }
        }
    }

    close(client);
    close(listener);
    return 0;
}
```

Client chat

```
#define _POSIX_C_SOURCE 200809L
#include <arpa/inet.h>
#include <errno.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/select.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#define LINE_MAX 4096
#define DATA_BUFSZ 8192

static void die(const char *msg) {
    perror(msg);
    exit(1);
}

static int send_all(int fd, const void *buf, size_t len) {
    const char *p = (const char*)buf;
    size_t sent = 0;
    while (sent < len) {
        ssize_t n = send(fd, p + sent, len - sent, 0);
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        sent += (size_t)n;
    }
    return 0;
}

static int send_line(int fd, const char *line) {
    return send_all(fd, line, strlen(line));
}

static int recv_line(int fd, char *out, size_t out_sz) {
```

```
    size_t used = 0;
    while (1) {
        if (used + 1 >= out_sz) return -2;
        char c;
        ssize_t n = recv(fd, &c, 1, 0);
        if (n == 0) return 0;
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        out[used++] = c;
        out[used] = '\0';
        if (c == '\n') return 1;
    }
}

static int connect_tcp(const char *ip, uint16_t port) {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    if (s < 0) return -1;

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    if (inet_pton(AF_INET, ip, &addr.sin_addr) != 1) {
        close(s);
        return -1;
    }
    if (connect(s, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        close(s);
        return -1;
    }
    return s;
}

static long long file_size(const char *path) {
    struct stat st;
    if (stat(path, &st) != 0) return -1;
    return (long long)st.st_size;
}

static int do_put(int control_fd, const char *server_ip, const char *path) {
    const char *fname = strrchr(path, '/');
    fname = fname ? fname + 1 : path;

    long long sz = file_size(path);
    if (sz < 0) {
        printf("PUT: can't stat file: %s\n", path);
        return 0;
    }

    char cmd[512];
    snprintf(cmd, sizeof(cmd), "STOR %s %lld\n", fname, sz);
    if (send_line(control_fd, cmd) < 0) {
        printf("PUT: failed to send STOR\n");
        return 0;
    }
```

```
    char resp[LINE_MAX];
    if (recv_line(control_fd, resp, sizeof(resp)) <= 0) {
        printf("PUT: no response\n");
        return 0;
    }

    // Expect: 150 PASV <port>
    unsigned pasv_port;
    if (sscanf(resp, "150 PASV %u", &pasv_port) != 1) {
        printf("PUT: server refused: %s", resp);
        return 0;
    }

    int data_fd = connect_tcp(server_ip, (uint16_t)pasv_port);
    if (data_fd < 0) {
        printf("PUT: can't connect data port %u\n", pasv_port);
        return 0;
    }

    FILE *fp = fopen(path, "rb");
    if (!fp) {
        close(data_fd);
        printf("PUT: can't open file\n");
        return 0;
    }

    char buf[DATA_BUFSZ];
    long long remaining = sz;
    while (remaining > 0) {
        size_t r = fread(buf, 1, (remaining > DATA_BUFSZ) ? DATA_BUFSZ : (size_t)remaining, fp);
        if (r == 0) break;
        if (send_all(data_fd, buf, r) < 0) {
            fclose(fp);
            close(data_fd);
            printf("PUT: send failed\n");
            return 0;
        }
        remaining -= (long long)r;
    }

    fclose(fp);
    close(data_fd);

    // Final status
    if (recv_line(control_fd, resp, sizeof(resp)) > 0) {
        printf("%s", resp);
        return (strncmp(resp, "226", 3) == 0);
    }
    printf("PUT: missing final reply\n");
    return 0;
}

static int do_get(int control_fd, const char *server_ip, const char *fname) {
    char cmd[512];
    snprintf(cmd, sizeof(cmd), "RETR %s\n", fname);
    if (send_line(control_fd, cmd) < 0) {
```

```
    printf("GET: failed to send RETR\n");
    return 0;
}

char resp[LINE_MAX];
if (recv_line(control_fd, resp, sizeof(resp)) <= 0) {
    printf("GET: no response\n");
    return 0;
}

// Expect: 150 PASV <port> <size>
unsigned pasv_port;
long long sz;
if (sscanf(resp, "150 PASV %u %lld", &pasv_port, &sz) != 2) {
    printf("GET: server refused: %s", resp);
    return 0;
}

int data_fd = connect_tcp(server_ip, (uint16_t)pasv_port);
if (data_fd < 0) {
    printf("GET: can't connect data port %u\n", pasv_port);
    return 0;
}

// Save as downloaded_<fname>
char outname[512];
snprintf(outname, sizeof(outname), "downloaded_%s", fname);

FILE *fp = fopen(outname, "wb");
if (!fp) {
    close(data_fd);
    printf("GET: can't open output file\n");
    return 0;
}

char buf[DATA_BUFSZ];
long long remaining = sz;
while (remaining > 0) {
    ssize_t chunk = (remaining > DATA_BUFSZ) ? DATA_BUFSZ : (ssize_t)remaining;
    ssize_t n = recv(data_fd, buf, chunk, 0);
    if (n <= 0) {
        fclose(fp);
        close(data_fd);
        printf("GET: data connection closed early\n");
        return 0;
    }
    fwrite(buf, 1, (size_t)n, fp);
    remaining -= n;
}

fclose(fp);
close(data_fd);

// Final status
if (recv_line(control_fd, resp, sizeof(resp)) > 0) {
    printf("%s", resp);
    if (strncmp(resp, "226", 3) == 0) {
```

```
            printf("Saved to %s (%lld bytes)\n", outname, sz);
            return 1;
        }
        return 0;
    }

    printf("GET: missing final reply\n");
    return 0;
}

int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "Usage: %s <server_ip> <port>\n", argv[0]);
        return 1;
    }
    const char *server_ip = argv[1];
    uint16_t port = (uint16_t)atoi(argv[2]);

    int control = connect_tcp(server_ip, port);
    if (control < 0) die("connect");

    printf("Connected to %s:%u\n", server_ip, port);
    printf("Commands:\n");
    printf("  (type any text) -> chat\n");
    printf("  /put <path>     -> upload file\n");
    printf("  /get <name>     -> download file\n");
    printf("  /quit           -> exit\n\n");

    // Read server greeting
    char line[LINE_MAX];
    if (recv_line(control, line, sizeof(line)) > 0) {
        printf("%s", line);
    }

    fd_set rfds;

    while (1) {
        FD_ZERO(&rfds);
        FD_SET(control, &rfds);
        FD_SET(STDIN_FILENO, &rfds);
        int maxfd = (control > STDIN_FILENO) ? control : STDIN_FILENO;

        int r = select(maxfd + 1, &rfds, NULL, NULL, NULL);
        if (r < 0) {
            if (errno == EINTR) continue;
            die("select");
        }

        // Incoming from server
        if (FD_ISSET(control, &rfds)) {
            int rr = recv_line(control, line, sizeof(line));
            if (rr == 0) {
                printf("Server disconnected.\n");
                break;
            }
            if (rr < 0) {
                printf("recv error.\n");
```

```
            break;
        }

        if (strncmp(line, "MSG ", 4) == 0) {
            printf("Server: %s", line + 4);
        } else {
            // status lines (220/226/5xx etc.)
            printf("%s", line);
        }
    }

    // User input
    if (FD_ISSET(STDIN_FILENO, &rfds)) {
        if (!fgets(line, sizeof(line), stdin)) break;

        if (strncmp(line, "/quit", 5) == 0) {
            send_line(control, "QUIT\n");
            break;
        } else if (strncmp(line, "/put ", 5) == 0) {
            char path[512];
            if (sscanf(line + 5, "%511s", path) == 1) {
                do_put(control, server_ip, path);
            } else {
                printf("Usage: /put <path>\n");
            }
        } else if (strncmp(line, "/get ", 5) == 0) {
            char fname[256];
            if (sscanf(line + 5, "%255s", fname) == 1) {
                do_get(control, server_ip, fname);
            } else {
                printf("Usage: /get <name>\n");
            }
        } else {
            // normal chat
            char out[LINE_MAX + 8];
            snprintf(out, sizeof(out), "MSG %s", line);
            if (send_line(control, out) < 0) {
                printf("Send failed.\n");
                break;
            }
        }
    }
  }

  close(control);
  return 0;
}
```

OUTPUT

```
root@BOOK-27C8PD5KBQ: ~,
root@BOOK-27C8PD5KBQ:~# cd ~/tcp_chat_ftp
root@BOOK-27C8PD5KBQ:~/tcp_chat_ftp# ls
client  mini_ftp_chat_client.c  mini_ftp_chat_server.c  server
root@BOOK-27C8PD5KBQ:~/tcp_chat_ftp# ./server 2121
Server listening on 0.0.0.0:2121
Chat: type and press Enter to send.
Commands received from client:
  MSG <text>\n
  STOR <name> <size>\n
  RETR <name>\n
  QUIT\n
Client connected.
Client: hi
Client wants upload: mini_ftp_chat_client.c (8617 bytes)
```

```
root@BOOK-27C8PD5KBQ: ~,
root@BOOK-27C8PD5KBQ:~# cd ~/tcp_chat_ftp
root@BOOK-27C8PD5KBQ:~/tcp_chat_ftp# ls
client  mini_ftp_chat_client.c  mini_ftp_chat_server.c  server
root@BOOK-27C8PD5KBQ:~/tcp_chat_ftp# ./client 127.0.0.1 2121
Connected to 127.0.0.1:2121
Commands:
  (type any text) -> chat
  /put <path>     -> upload file
  /get <name>     -> download file
  /quit           -> exit

220 Welcome to mini-FTP-chat server
hi
/put  mini_ftp_chat_client.c
226 Stored mini_ftp_chat_client.c
```

(D) IMPLEMENTING SECURITY PROTOCOL/CRYPTOGRAPHY ALGORITHM FOR SECURE DATA COMMUNICATION

ALGORITHM:

Key Generation

1. Start
2. Generate Ed25519 private key sk
3. Derive public key pk = sk.public_key()
4. Save private key to file server_signing_key.pem in PEM (PKCS8) format (no encryption)
5. Save public key to file server_signing_pub.pem in PEM format
6. Print confirmation message
7. Stop

Secure TCP Server

1. Start
2. Load server Ed25519 private signing key from server_signing_key.pem
3. Create TCP socket, bind to (HOST, PORT), listen, and accept client connection

4. Handshake (Server → Client)
5. Set counters:
    - recv_counter = -1 (anti-replay)
    - send_counter = 0
6. Secure Messaging Loop
7. Close client socket and server socket
8. Stop

Secure TCP Client

1. Start
2. Load server Ed25519 public key from server_signing_pub.pem
3. Create TCP socket and connect to (SERVER_HOST, SERVER_PORT)
4. Handshake (Server → Client)
5. Compute session secrets
6. Set counters:
    - send_counter = 0
    - recv_counter = -1
7. Secure Messaging Loop
8. Close socket
9. Stop

CODE:
gen_server_key

```
from cryptography.hazmat.primitives.asymmetric import ed25519
from cryptography.hazmat.primitives import serialization

sk = ed25519.Ed25519PrivateKey.generate()
pk = sk.public_key()

with open("server_signing_key.pem", "wb") as f:
    f.write(sk.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    ))

with open("server_signing_pub.pem", "wb") as f:
    f.write(pk.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    ))

print("Generated server_signing_key.pem and server_signing_pub.pem")
```

secure_server

```
import os, json, socket, struct
from cryptography.hazmat.primitives.asymmetric import x25519, ed25519
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

HOST = "127.0.0.1"
```

```
PORT = 4444

def send_frame(conn, obj: dict):
    data = json.dumps(obj).encode()
    conn.sendall(struct.pack("!I", len(data)) + data)

def recv_frame(conn) -> dict:
    hdr = conn.recv(4)
    if not hdr:
        raise ConnectionError("closed")
    (n,) = struct.unpack("!I", hdr)
    data = b""
    while len(data) < n:
        chunk = conn.recv(n - len(data))
        if not chunk:
            raise ConnectionError("closed")
        data += chunk
    return json.loads(data.decode())

def load_server_signing_key():
    with open("server_signing_key.pem", "rb") as f:
        return serialization.load_pem_private_key(f.read(), password=None)

def hkdf_key(shared_secret: bytes, salt: bytes) -> bytes:
    return HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=salt,
        info=b"secure-socket-demo-v1",
    ).derive(shared_secret)

def main():
    signing_sk = load_server_signing_key()

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind((HOST, PORT))
    s.listen(1)
    print(f"[server] listening on {HOST}:{PORT}")

    conn, addr = s.accept()
    print("[server] client connected:", addr)

    # --- Handshake: server ECDH ephemeral + signature ---
    server_eph_sk = x25519.X25519PrivateKey.generate()
    server_eph_pk = server_eph_sk.public_key().public_bytes(
        encoding=serialization.Encoding.Raw,
        format=serialization.PublicFormat.Raw
    )
    server_nonce = os.urandom(16)

    signature = signing_sk.sign(server_eph_pk + server_nonce)

    send_frame(conn, {
        "type": "SERVER_HELLO",
        "server_eph_pk": server_eph_pk.hex(),
        "server_nonce": server_nonce.hex(),
        "sig": signature.hex()
```

```
})

msg = recv_frame(conn)
if msg.get("type") != "CLIENT_HELLO":
    raise ValueError("bad handshake")

client_eph_pk = bytes.fromhex(msg["client_eph_pk"])
client_nonce = bytes.fromhex(msg["client_nonce"])

client_pub = x25519.X25519PublicKey.from_public_bytes(client_eph_pk)
shared = server_eph_sk.exchange(client_pub)

salt = server_nonce + client_nonce
key = hkdf_key(shared, salt)
aead = AESGCM(key)

print("[server] handshake complete, secure channel ready")

recv_counter = -1
send_counter = 0

while True:
    pkt = recv_frame(conn)
    if pkt.get("type") != "DATA":
        continue

    nonce = bytes.fromhex(pkt["nonce"])
    ct = bytes.fromhex(pkt["ciphertext"])
    counter = pkt["counter"]

    # replay protection
    if counter <= recv_counter:
        print("[server] replay detected, dropping")
        continue
    recv_counter = counter

    try:
        pt = aead.decrypt(nonce, ct, None)
    except Exception:
        print("[server] auth failed / tampered packet")
        continue

    text = pt.decode(errors="replace")
    print(f"[client] {text}")

    if text.strip().lower() == "quit":
        print("[server] closing")
        break

    reply = f"ACK: {text}".encode()
    # 12-byte nonce for AESGCM
    out_nonce = send_counter.to_bytes(12, "big")
    out_ct = aead.encrypt(out_nonce, reply, None)

    send_frame(conn, {
        "type": "DATA",
        "counter": send_counter,
```

```
            "nonce": out_nonce.hex(),
            "ciphertext": out_ct.hex()
        })
        send_counter += 1

    conn.close()
    s.close()

if __name__ == "__main__":
    main()
```

secure_client

```
import os, json, socket, struct
from cryptography.hazmat.primitives.asymmetric import x25519, ed25519
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

SERVER_HOST = "127.0.0.1"
SERVER_PORT = 4444

def send_frame(conn, obj: dict):
    data = json.dumps(obj).encode()
    conn.sendall(struct.pack("!I", len(data)) + data)

def recv_frame(conn) -> dict:
    hdr = conn.recv(4)
    if not hdr:
        raise ConnectionError("closed")
    (n,) = struct.unpack("!I", hdr)
    data = b""
    while len(data) < n:
        chunk = conn.recv(n - len(data))
        if not chunk:
            raise ConnectionError("closed")
        data += chunk
    return json.loads(data.decode())

def load_server_pubkey():
    with open("server_signing_pub.pem", "rb") as f:
        return serialization.load_pem_public_key(f.read())

def hkdf_key(shared_secret: bytes, salt: bytes) -> bytes:
    return HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=salt,
        info=b"secure-socket-demo-v1",
    ).derive(shared_secret)

def main():
    server_verify_pk = load_server_pubkey()

    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((SERVER_HOST, SERVER_PORT))
    print(f"[client] connected to {SERVER_HOST}:{SERVER_PORT}")
```

```
# --- Receive SERVER_HELLO and verify signature ---
msg = recv_frame(sock)
if msg.get("type") != "SERVER_HELLO":
    raise ValueError("bad handshake")

server_eph_pk = bytes.fromhex(msg["server_eph_pk"])
server_nonce = bytes.fromhex(msg["server_nonce"])
sig = bytes.fromhex(msg["sig"])

# Verify authenticity of server ephemeral key
try:
    server_verify_pk.verify(sig, server_eph_pk + server_nonce)
except Exception:
    raise RuntimeError("SERVER AUTH FAILED (possible MITM)")

# --- Send CLIENT_HELLO ---
client_eph_sk = x25519.X25519PrivateKey.generate()
client_eph_pk = client_eph_sk.public_key().public_bytes(
    encoding=serialization.Encoding.Raw,
    format=serialization.PublicFormat.Raw
)
client_nonce = os.urandom(16)

send_frame(sock, {
    "type": "CLIENT_HELLO",
    "client_eph_pk": client_eph_pk.hex(),
    "client_nonce": client_nonce.hex()
})

server_pub = x25519.X25519PublicKey.from_public_bytes(server_eph_pk)
shared = client_eph_sk.exchange(server_pub)

salt = server_nonce + client_nonce
key = hkdf_key(shared, salt)
aead = AESGCM(key)

print("[client] handshake complete, secure channel ready")
print("Type messages. 'quit' to end.\n")

send_counter = 0
recv_counter = -1

while True:
    text = input("You: ").strip()
    pt = text.encode()

    nonce = send_counter.to_bytes(12, "big")
    ct = aead.encrypt(nonce, pt, None)

    send_frame(sock, {
        "type": "DATA",
        "counter": send_counter,
        "nonce": nonce.hex(),
        "ciphertext": ct.hex()
    })
    send_counter += 1
```

```
        if text.lower() == "quit":
            break

        # wait for server reply
        resp = recv_frame(sock)
        if resp.get("type") != "DATA":
            continue

        counter = resp["counter"]
        if counter <= recv_counter:
            print("[client] replay detected, dropping")
            continue
        recv_counter = counter

        r_nonce = bytes.fromhex(resp["nonce"])
        r_ct = bytes.fromhex(resp["ciphertext"])

        try:
            r_pt = aead.decrypt(r_nonce, r_ct, None)
            print("Server:", r_pt.decode(errors="replace"))
        except Exception:
            print("[client] auth failed / tampered packet")

    sock.close()

if __name__ == "__main__":
    main()
```

```
root@BOOK-27C8PD5KBQ:~# python3 -c "import cryptography; print('cryptography OK')"
cryptography OK
root@BOOK-27C8PD5KBQ:~# mkdir -p ~/secure_socket_demo
root@BOOK-27C8PD5KBQ:~# cd ~/secure_socket_demo
root@BOOK-27C8PD5KBQ:~/secure_socket_demo# nano gen_server_key.py
root@BOOK-27C8PD5KBQ:~/secure_socket_demo# python3 gen_server_key.py
Generated server_signing_key.pem and server_signing_pub.pem
root@BOOK-27C8PD5KBQ:~/secure_socket_demo# ls -l
total 12
-rw-r--r-- 1 root root 685 Jan  8 16:20 gen_server_key.py
-rw-r--r-- 1 root root 119 Jan  8 16:20 server_signing_key.pem
-rw-r--r-- 1 root root 113 Jan  8 16:20 server_signing_pub.pem
root@BOOK-27C8PD5KBQ:~/secure_socket_demo# nano secure_server.py
root@BOOK-27C8PD5KBQ:~/secure_socket_demo# nano secure_client.py
root@BOOK-27C8PD5KBQ:~/secure_socket_demo# python3 secure_server.py
[server] listening on 127.0.0.1:4444
[server] client connected: ('127.0.0.1', 39158)
[server] handshake complete, secure channel ready
[client] HI
[client] HOW ARE YOU
```

```
root@BOOK-27C8PD5KBQ:~# cd ~/secure_socket_demo
root@BOOK-27C8PD5KBQ:~/secure_socket_demo# python3 secure_client.py
[client] connected to 127.0.0.1:4444
[client] handshake complete, secure channel ready
Type messages. 'quit' to end.

You: HI
Server: ACK: HI
You: HOW ARE YOU
Server: ACK: HOW ARE YOU
You:
```